SHERLOCK

Security Review For Sodax

Collaborative Audit Prepared For:	**Sodax**
Lead Security Expert(s):	**defsec** **TheBlockChainer**
Date Audited:	**November 11 - November 15, 2025**

Introduction

The audit covers all core near contracts built by the Sodax protocol, including:

- Relay Connection Contracts : for managing cross-chain messaging and execution
- Asset Manager Contracts : for handling assets and execution flows
- Rate Limiter Contracts : for enforcing transaction and withdrawal limits

Scope

Repository: icon-project/sodax-contracts

Audited Commit: 4575b72344d2ab96e0b2fad7f0d49f92bd6bb29e

Final Commit: c5cd641f704bade683663a3870dcf9367cf909d0

Files:

- near/asset-manager/src/lib.rs
- near/asset-manager/src/transfer.rs
- near/fungible-token/src/lib.rs
- near/fungible-token/tests/storage.rs
- near/fungible-token/tests/supply.rs
- near/fungible-token/tests/transfer.rs
- near/intent-filler/src/fill.rs
- near/intent-filler/src/lib.rs
- near/rate-limit/src/lib.rs
- near/rate-limit/src/payload.rs
- near/router/src/lib.rs
- near/router/src/state.rs

Final Commit Hash

c5cd641f704bade683663a3870dcf9367cf909d0

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
2	7	8

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: No replay protection in rate limiter signatures [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/14

Summary

The Rate Limiter contract's `recv_message` function lacks replay protection for signed administrative messages. Valid signed messages can be replayed multiple times within their deadline window, allowing repeatedly execute operations such as resetting rate limits, reverting configuration changes, and manipulating contract state. The only protection is a deadline timestamp check, which permits unlimited replays until expiration.

Vulnerability Detail

The issue exists in the message verification flow where signed messages are validated but not tracked for replay prevention. The Message struct contains only three fields:

```
pub struct Message {
    pub kind: u8,
    pub deadline: u64,
    pub data: Actions,
}
```

Notably absent is any nonce, sequence number, or unique identifier that would prevent message reuse.

The `decode_verify_message` function performs signature verification but implements only a single temporal check:

```
fn decode_verify_message(&self, payload: Vec<u8>) -> (Message, Vec<u8>) {
    let payload_decoded = payload::Payload::from_bytes(&payload).unwrap();
    let message = Message::from_bytes(&payload_decoded.data).unwrap();

    let now_sec = env::block_timestamp_ms() / 1000;
    require!(now_sec < message.deadline, "Expired");  // ONLY CHECK

    let sender = self
        .recover_signer(
            &env::keccak256(payload_decoded.data.as_ref()),
            &payload_decoded.signature,
        )
        .unwrap();
    return (message, sender);
}
```

The `recv_message` function then executes the action without any state tracking:

```
pub fn recv_message(&mut self, src_chain_id: u128, src_address: Vec<u8>, payload:
↪  Vec<u8>) {
    self.assert_connection();
    require!(src_chain_id == self.state.hub_chain_id, "Bad chain");
    require!(src_address == self.state.hub_manager, "Bad manager");

    let (message, sender) = self.decode_verify_message(payload);
    let action = message.data;

    match action {
        payload::Actions::SetHubAdmin { admin } => {
            self.assert_hub_admin(&sender);
            self.state.hub_admin = admin;
        }
        payload::Actions::SetHubSigners { signers } => {
            self.assert_hub_admin(&sender);
            self.state.hub_signers = signers;
        }
        payload::Actions::SetRateLimit { token, rate_per_second,
        ↪  max_available_withdraw } => {
            self.assert_hub_admin_or_signer(&sender);
            let token = String::from_utf8(token).unwrap();
            self.set_rate_limit_inner(token, rate_per_second,
            ↪  max_available_withdraw);
        }
        payload::Actions::ResetRateLimit { token } => {
            self.assert_hub_admin_or_signer(&sender);
            let token = String::from_utf8(token).unwrap();
            let mut cfg = self.rate_limits.get(&token).cloned().expect("No such
            ↪  token");
            let now_sec = env::block_timestamp_ms() / 1000;
            cfg.last_updated = now_sec;
            cfg.available = cfg.max_available;  // replay
            self.rate_limits.insert(token, cfg);
        }
        payload::Actions::Pause {} => {
            self.assert_hub_admin_or_signer(&sender);
            self.state.paused = true;
        }
    }
}
```

Impact

Valid admin messages can be replayed.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/rate-limit/src/lib.rs#L285

Tool Used

Manual Review

Recommendation

Implement replay protection using nonce-based tracking.

Discussion

ibrizsabin

@defsec same as other replay attacks .

defsec

The issue is fixed however, whenever messages have same deadline, that will block messages with same deadline.

The team comment :

```
ya i had the similar concern and discussed with the team as well but we think it
s a trivial issue for now since we will be the ones crafting the messages and we
can make sure that they dont collide.
```

Issue H-2: Broken Ownership Check in RateLimiter Leads to Permanently Inaccessible Admin Functions [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/31

Summary

The `RateLimiter` contract implements a flawed ownership check that compares `env::predecessor_account_id()` with `env::current_account_id()`, making all admin functions permanently inaccessible in normal deployment scenarios. This prevents critical operations like unpausing the contract, updating rate limits, and modifying contract addresses.

Vulnerability Detail

The `assert_owner()` function in the `RateLimiter` contract checks if the caller is the contract itself:

```
fn assert_owner(&self) {
    require!(
        env::predecessor_account_id() == env::current_account_id(),
        "Only contract owner"
    );
}
```

This check only passes when `predecessor_account_id` equals `current_account_id`, which means the contract must call itself. Unlike the `SpokeAssetsManager` contract which stores an `owner: AccountId` field and checks against it, the `RateLimiter` has no owner field and uses this self-referential check.

In a normal deployment:

- Contract deployed to `ratelimit.near`
- Admin account is `admin.near`
- When admin calls `pause()`: `predecessor = admin.near`, `current_account_id = ratelimit.near`
- Check fails: `admin.near != ratelimit.near` ❌

The unit tests pass because they artificially set both values to the same account:

```
fn get_context(predecessor_account_id: AccountId) -> VMContextBuilder {
    builder
        .current_account_id(accounts(0))
```

```
        .predecessor_account_id(predecessor_account_id);
}

#[test]
fn owner_can_pause_unpause() {
    let context = get_context(accounts(0)); // Both set to accounts(0)
    c.pause(); // Passes artificially
}
```

Impact

The broken ownership check renders the `RateLimiter` contract permanently unadministrable after deployment. No external account can satisfy the condition `predecessor_account_id == current_account_id`, effectively locking all admin functions.

Immediate consequences:

1. **Rate limits cannot be configured**
2. **Contract can be permanently frozen**
3. **Critical addresses cannot be updated**

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/rate-limit/src/lib.rs#L109-L114

```
fn assert_owner(&self) {
    require!(
        env::predecessor_account_id() == env::current_account_id(),
        "Only contract owner"
    );
}
```

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/rate-limit/src/lib.rs#L216-L229

```
pub fn unpause(&mut self) {
    self.assert_owner();
    self.state.paused = false;
}

pub fn set_asset_manager(&mut self, asset_manager: AccountId) {
    self.assert_owner();
    self.state.asset_manager = asset_manager;
}
```

```
pub fn set_connection(&mut self, connection: AccountId) {
    self.assert_owner();
    self.state.connection = connection;
}
```

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/rate-limit/src/lib.rs#L247-L255

```
pub fn set_rate_limit(
    &mut self,
    token: String,
    rate_per_second: u128,
    max_allowed_withdrawal: u128,
) {
    self.assert_owner();
    self.set_rate_limit_inner(token, rate_per_second, max_allowed_withdrawal);
}
```

Tool Used

Manual Review

Recommendation

Follow the same ownership pattern used in `SpokeAssetsManager`:

```
#[near(contract_state)]
#[derive(PanicOnDefault)]
pub struct RateLimiter {
    pub state: State,
    pub owner: AccountId,  // Add owner field
    pub rate_limits: LookupMap<String, RateLimitConfig>,
}

#[near]
impl RateLimiter {
    #[init]
    pub fn new(
        owner: AccountId,  // Add owner parameter
        asset_manager: AccountId,
        connection: AccountId,
        // ... other parameters
    ) -> Self {
        Self {
            state: State {
                // ... initialize state
            },
```

```
            owner,  // Store owner
            rate_limits: LookupMap::new(b"r"),
        }
    }
}

fn assert_owner(&self) {
    require!(
        env::predecessor_account_id() == self.owner,  // Check against stored owner
        "Only contract owner"
    );
}

// Add owner transfer function
pub fn change_owner(&mut self, new_owner: AccountId) {
    self.assert_owner();
    self.owner = new_owner;
}
```

Discussion

ibrizsabin

@vendrell46 Thank you for noticing the issue. Acknowledged.

vendrell46

Fixed.

Issue M-1: Router contract enables arbitrary fund extraction by whitelisted accounts [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/15

Summary

The Router contract allows whitelisted accounts to execute arbitrary cross-contract calls with user-controlled parameters including target contract, method name, arguments, gas, and deposit amounts.

Vulnerability Detail

While caller whitelisting provides some protection, it creates a powerful execution primitive that can drain all router funds if any whitelisted account is compromised or malicious.

There are no restrictions on:

- Target contract addresses (can call any contract)
- Deposit amounts (can send all router balance)
- Number of calls in sequence (unlimited)
- Arguments passed (user-controlled with template substitution)

Impact

Compromised whitelisted account = full drain.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/router/src/lib.rs#L50

Tool Used

Manual Review

Recommendation

Implement target contract whitelist.

Discussion

ibrizsabin

@defsec ya that is by design because router contract will not be exposed to any other party other than our own services. It is there only for our solvers so that they can batch multiple actions via it. So its kind of like a trusted setup.

defsec

Acknowledged.

Issue M-2: User fees not refunded on cross-chain message failure [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/16

Summary

The `transfer()` function accepts attached deposit of `amount + fee`, where the fee is forwarded to the connection contract for cross-chain message delivery. However, when message delivery fails and the transfer is refunded, only the `amount` is returned to the user. The fee remains permanently locked in the asset manager contract, causing financial loss for every failed transfer.

Vulnerability Detail

The fee is attached to `connection.send_message()` via:

```
.with_attached_deposit(NearToken::from_yoctonear(fee))
```

When the connection contract call fails, Near's execution model refunds unused attached deposits back to the calling contract (asset_manager), not to the original user. Therefore, fee goes to connection contract if call succeeds.

Impact

Every failed transfer loses fee.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/lib.rs#L122

Tool Used

Manual Review

Recommendation

Consider refunding full amount including fee.

Discussion

ibrizsabin

Acknowledged.

defsec

Fix is confirmed.

Issue M-3: Router contract enables unauthorized fund drainage through missing deposit validation [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/18

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The Router contract's `execute_calls` function allows whitelisted users to execute NEAR token transfers using user-specified deposit amounts without validating that these amounts match the actual attached deposit. This design flaw enables whitelisted users to drain the entire NEAR balance held by the router contract by specifying deposit amounts that exceed their payment, effectively stealing funds from the contract's accumulated balance.

Vulnerability Detail

The issue stems from a fundamental mismatch between two separate value sources in the contract's execution flow. The `execute_calls` function accepts an attached deposit through the `#[payable]` attribute, which can be queried via `env::attached_deposit()`. However, the function also accepts a vector of `Call` structures, each containing a `deposit` field that specifies how much NEAR should be transferred.

The flaw is that `execute_calls` performs no validation to ensure that the sum of all `call.deposit` values matches or is less than the `attached_deposit`. The function simply stores the calls and immediately proceeds to execution via `dispatch_next`.

When `dispatch_next` processes transfer-type calls, it extracts the `deposit` value directly from the `Call` structure and uses it in a `Promise::transfer()` call. In the NEAR protocol, `Promise::transfer()` transfers funds from the contract's own balance, not from attached deposits. Attached deposits remain separate and must be explicitly handled by the contract logic.

The `get_remaining_deposit` function attempts to calculate remaining deposits using `saturating_sub`, but this only affects the callback deposit amount, not the actual transfer amount. When `call.deposit` exceeds `attached_deposit`, the subtraction saturates to zero, but the transfer still proceeds using the full `call.deposit` value from the contract's balance.

This creates a scenario where a whitelisted user can attach minimal or zero NEAR to the function call while specifying arbitrarily large deposit amounts in the Call structures, causing the contract to transfer funds from its own accumulated balance rather than from the user's payment.

Impact

Any whitelisted user can drain the entire contract balance in a single transaction by specifying a deposit amount equal to or greater than the contract's balance.

Code Snippet

```
#[payable]
pub fn execute_calls(&mut self, calls: Vec<Call>) -> Promise {
    let caller = env::predecessor_account_id();
    assert!(
        self.whitelist.contains(&caller),
        "{}",
        ContractError::NotWhitelisted(caller)
    );
    if calls.is_empty() {
        return Promise::new(env::current_account_id());
    }
    let request_id = self.get_next_request_id();
    let stack = Stack::new(calls);
    self.last_reply.insert(request_id, LastReply::Empty);
    self.stacks.insert(request_id, stack);
    return self.dispatch_next(request_id);
}
```

Tool Used

Manual Review

Recommendation

The contract must validate that the sum of all requested deposits does not exceed the attached deposit before processing any calls. This validation should occur in the `execute_calls` function immediately after the whitelist check.

Discussion

ibrizsabin

@defsec Thats a valid concern but router contract is not supposed to have any funds in first place. Still it does make sense to have that validation to prevent incorrect calls. I acknowledge the issue but i would flag this as medium there should not be any funds in router contract to drain. The whitelisted users are mostly our own solver services so the contract is kind of private as well.

defsec

Fix is confirmed.

Issue M-4: `add_whitelist` and `remove_whitelist` are marked payable but don't use attached deposit [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/21

Summary

The `add_whitelist` and `remove_whitelist` functions are marked with `#[payable]` but don't use `env::attached_deposit()` or transfer any NEAR. This is unnecessary and can lead to accidental fund loss if users attach NEAR when calling these functions, thinking payment is required.

Vulnerability Detail

In NEAR Protocol, the `#[payable]` attribute is required when a function needs to receive NEAR tokens. However, these administrative functions only:

1. Check ownership
2. Insert/remove accounts from a `LookupSet`

They don't:

- Use `env::attached_deposit()`
- Transfer NEAR
- Need any payment

Impact

sers may attach NEAR thinking it's required, losing funds.

Code Snippet

```
#[payable]  //  Unnecessary - function doesn't use attached_deposit
pub fn add_whitelist(&mut self, account_id: AccountId) {
    self.assert_owner();
    self.whitelist.insert(account_id);  // Just modifies a set, no NEAR needed
}

#[payable]  //  Unnecessary - function doesn't use attached_deposit
pub fn remove_whitelist(&mut self, account_id: AccountId) {
    self.assert_owner();
```

```
    self.whitelist.remove(&account_id);  // Just modifies a set, no NEAR needed
}
```

Tool Used

Manual Review

Recommendation

Remove the `#[payable]` attribute from both functions.

Discussion

ibrizsabin

Acknowledged. Thank you.

defsec

The issue is not fixed, however now privilege functions are needing one yocto.

Issue M-5: Rate limiter allows unlimited withdrawals for tokens without rate limits [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/23

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The Rate Limiter's `verify_withdraw` function returns early without any validation if a token doesn't have a rate limit configured in the `rate_limits` map. This allows unlimited withdrawals for any token that hasn't been explicitly rate-limited, enabling attackers to drain funds by exploiting unconfigured tokens or tokens that had their rate limits removed.

Vulnerability Detail

The `verify_withdraw` function in the Rate Limiter contract is called by the Asset Manager for every cross-chain transfer to enforce rate limits. However, the function contains an early return that bypasses all validation when a token doesn't have a rate limit configuration:

```
pub fn verify_withdraw(&mut self, token: String, amount: U128) {
    require!(!self.state.paused, "Contract paused");
    self.assert_asset_manager();
    let amt: u128 = amount.into();
    if self.rate_limits.get(&token).is_none() {
        return;  // @audit Returns without any validation!
    }
    // Rate limit checks only execute if token exists in rate_limits map
    let mut cfg = self.rate_limits.get(&token).cloned().unwrap();
    let avail = self.compute_available(&mut cfg);
    require!(avail >= amt, "Rate limit exceeded");
    cfg.available = avail - amt;
    self.rate_limits.insert(token.to_string(), cfg);
}
```

Impact

Tokens without rate limits can be withdrawn in any amount, regardless of size.

Code Snippet

```
pub fn verify_withdraw(&mut self, token: String, amount: U128) {
    require!(!self.state.paused, "Contract paused");
    self.assert_asset_manager();
    let amt: u128 = amount.into();

    // @audit If token has no rate limit, returns without any checks
    if self.rate_limits.get(&token).is_none() {
        return;  // Unlimited withdrawals allowed!
    }

    // @audit Rate limit checks only happen if token exists
    let mut cfg = self.rate_limits.get(&token).cloned().unwrap();
    let avail = self.compute_available(&mut cfg);
    require!(avail >= amt, "Rate limit exceeded");
    cfg.available = avail - amt;
    self.rate_limits.insert(token.to_string(), cfg);
}
```

Call Site (`near/asset-manager/src/lib.rs:188-190`):

```
let rate_limit_promise = rate_limit_trait::ext(self.state.rate_limit.clone())
    .with_unused_gas_weight(1)
    .verify_withdraw(token_str, U128(transfer.amount));
// If verify_withdraw returns early, the promise succeeds and transfer proceeds
```

Tool Used

Manual Review

Recommendation

Modify `verify_withdraw` to require that all tokens have rate limits configured or if the design requires allowing tokens without explicit rate limits, enforce a default restrictive rate limit.

Discussion

ibrizsabin

@defsec currently it is by design . There will be ratelimits for token officially registered in our platform but for other tokens it will be open. Hence we can say its a non-issue right now.

defsec

Acknowledged.

Issue M-6: Zero-fee transfer in asset manager [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/32

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The NEAR Asset Manager's `transfer` function allows users to send cross-chain transfers with zero fees by setting `attached_deposit` equal to the transfer `amount`. The validation only checks `attached_deposit >= amount`, which permits equality, resulting in `fee = 0`.

Vulnerability Detail

The `transfer` function calculates the fee as `attached_deposit - amount` but does not enforce a minimum fee requirement. The validation `attached_deposit >= NearToken::from_yoctonear(amount)` allows equality, meaning users can send transfers where the attached deposit exactly equals the transfer amount, resulting in zero fees.

Impact

Users can send transfers without paying fees.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/lib.rs#L122

Tool Used

Manual Review

Recommendation

Add a minimum fee constant and validate that the fee meets this requirement.

Discussion

vendrell46

I was considering this as well, but I saw it on the EVM side, which made me think that for the native token, a zero fee is allowed.

```
// For native token transfers, forward only the excess msg.value as fee
uint256 messageFee = token == NATIVE_TOKEN ? msg.value - amount : msg.value;
```

And if it's not allowed, and my understanding is correct, then do they have a bug on the EVM, then?

ibrizsabin

@vendrell46 @defsec ya by design we do allow zero fees. we can later filter such messages on relay side if needed. That is the idea. We can flag this as non-issue.

Issue M-7: Fees not refunded on cross-chain message failure [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/36

Summary

When cross-chain message delivery fails in the NEAR Asset Manager contract, users only receive the transfer amount back, but the fee they paid is permanently lost. The fee is calculated from the attached deposit but is not stored in the Transfer struct, and therefore cannot be refunded in the failure callback. The fee remains locked in the asset manager contract balance.

Vulnerability Detail

The `transfer()` function accepts an attached deposit that should cover both the transfer amount and a fee for cross-chain message delivery. However, when the message delivery fails, only the transfer amount is refunded to the user, while the fee remains in the contract.

Fee Collection (line 122):

```
let fee = attached_deposit.as_yoctonear()-amount;
```

The fee is calculated but never stored in the Transfer struct:

```
let transfer = Transfer {
    token: NEAR_TOKEN.as_bytes().to_vec(),
    from: sender.as_str().as_bytes().to_vec(),
    to: to,
    amount,  // @audit Only amount stored, fee not stored
    data: data,
};
```

The fee is then attached to the connection contract call:

```
connection_trait::ext(self.state.connection.clone())
    .with_attached_deposit(NearToken::from_yoctonear(deposit))  // @audit fee is
    ↪  passed here
    .send_message(...)
```

Incomplete Refund (line 294): When the connection contract call fails, NEAR's execution model automatically refunds the unused attached deposit (the fee) back to the asset manager contract, NOT to the original user. The callback then only refunds the stored `transfer.amount`:

```
PromiseResult::Failed => {
    let transfer = self.pending_transfers.remove(&transfer_id).unwrap();
    let recepient = Self::account_from_bytes(&transfer.from).unwrap();
    return PromiseOrValue::Promise(
        Promise::new(recepient)
            .transfer(NearToken::from_yoctonear(transfer.amount)),  // @audit Only
            ↪  amount refunded
    );
}
```

Impact

Every failed cross-chain transfer causes users to permanently lose their fee.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/lib.rs#L129

Tool Used

Manual Review

Recommendation

In the `transfer()` function, after calculating the fee from `attached_deposit - amount`, store this fee value in the Transfer struct when creating the transfer object. This ensures the fee is preserved for potential refund scenarios.

Discussion

ibrizsabin

Acknowledged.

defsec

The issue is fixed with https://github.com/icon-project/sodax-contracts/pull/557/files.

Issue L-1: No owner transfer safety in change_owner [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/13

Summary

The `change_owner` function immediately transfers ownership without confirmation. If the new owner address is incorrect or inaccessible, **ownership is permanently lost**.

Vulnerability Detail

The issue exists in the change_owner function which directly assigns the new owner without any safety checks or two-step verification process:

```
pub fn change_owner(&mut self, new_owner: AccountId) {
    self.assert_owner();
    self.owner = new_owner;  // Immediate transfer, no confirmation!
}
```

The function flow is:

1. Current owner calls `change_owner(new_address)`
2. Function validates current owner via `assert_owner()`
3. Ownership is immediately and permanently transferred to new_address
4. No verification that `new_address` is accessible or correct

Impact

If ownership is transferred to an incorrect or inaccessible address, all admin functions become permanently unusable.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/lib.rs#L330

Tool Used

Manual Review

Recommendation

Implement a two-step ownership transfer pattern (also known as ”propose-and-accept” or ”pending ownership”) that requires the new owner to actively claim ownership.

Discussion

ibrizsabin

@defsec Acknowledged.

vendrell46

Fixed

Issue L-2: Asset manager missing fee validation for ft transfers [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/17

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `ft_on_transfer` callback sends cross-chain messages with zero fees, bypassing the fee mechanism that applies to native NEAR transfers.

Vulnerability Detail

When users transfer fungible tokens via `ft_transfer_call`, the asset manager processes the transfer but sends the cross-chain message with zero fees:

```
#[near]
impl FungibleTokenReceiver for SpokeAssetsManager {
    fn ft_on_transfer(
        &mut self,
        sender_id: AccountId,
        amount: U128,
        msg: String,
    ) -> PromiseOrValue<U128> {
        // ... create transfer ...
        return PromiseOrValue::Promise(
            Self::ext(env::current_account_id())
                .with_unused_gas_weight(1)
                .connection_send_message(payload, transfer_id, 0),  // @audit Zero
                ↪  fee
        );
    }
}
```

Comparison with Native NEAR Transfer:

In the `transfer` function, fees are calculated and passed:

```
let fee = attached_deposit.as_yoctonear()-amount;
self.connection_send_message(payload, transfer_id, fee)  // @audit Fee included
```

Impact

Users can avoid paying cross-chain fees by using FT tokens instead of NEAR.

Tool Used

Manual Review

Recommendation

Implement fee collection for FT transfers.

Discussion

ibrizsabin

@defsec ya it is by design right now because there will be additional call backs and verifications if want to take fee in fungible token we decided to leave it as zero fees. Also most probably we will only be dealing with NEAR token for forseeable future we decided to let it be zero fees for now. I do acknowledge the issue.

Issue L-3: Missing `assert_one_yocto` protection on privileged functions allows silent execution via compromised Function Access Keys [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/20

Summary

The asset manager's privileged functions `set_rate_limit`, `set_connection`, `set_hub_config`, `change_owner` lack `assert_one_yocto()` protection, allowing silent execution via Function Access Keys if the frontend is compromised.

Vulnerability Detail

In NEAR, there are two types of access keys:

- **Full Access Keys**: Require user signature confirmation for every transaction
- **Function Access Keys**: Can call whitelisted functions without user confirmation (commonly used by frontends for better UX)

The vulnerable functions only check `[assert_owner()` but do not enforce `assert_one_yocto()`, which means they can be called using Function Access Keys without the owner's explicit confirmation.

Impact

If a Function Access Key is granted to a compromised frontend, an attacker can silently hijack the protocol configuration by changing critical contract addresses (connection, rate_limit, hub) or transfer ownership to take complete control of the contract. This disrupts all in-flight transfers and protocol operations without the owner receiving any transaction signature prompt, allowing significant damage before detection.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/lib.rs#L314-L333

```
pub fn set_rate_limit(&mut self, rate_limit: AccountId) {
    self.assert_owner();  // No assert_one_yocto!
    self.state.rate_limit = rate_limit;
}

pub fn set_connection(&mut self, connection: AccountId) {
```

```
    self.assert_owner();  // No assert_one_yocto!
    self.state.connection = connection;
}

pub fn set_hub_config(&mut self, hub_chain_id: u128, hub_asset_manager: Vec<u8>) {
    self.assert_owner();  // No assert_one_yocto!
    self.state.hub_chain_id = hub_chain_id;
    self.state.hub_asset_manager = hub_asset_manager;
}

pub fn change_owner(&mut self, new_owner: AccountId) {
    self.assert_owner();  // No assert_one_yocto!
    self.owner = new_owner;
}
```

Tool Used

Manual Review

Recommendation

Add `#[payable]` and `assert_one_yocto()` to all privileged functions to force Full Access Key usage:

```
#[payable]
pub fn set_rate_limit(&mut self, rate_limit: AccountId) {
    assert_one_yocto();  // Forces Full Access Key usage
    self.assert_owner();
    self.state.rate_limit = rate_limit;
}

#[payable]
pub fn set_connection(&mut self, connection: AccountId) {
    assert_one_yocto();
    self.assert_owner();
    self.state.connection = connection;
}

#[payable]
pub fn set_hub_config(&mut self, hub_chain_id: u128, hub_asset_manager: Vec<u8>) {
    assert_one_yocto();
    self.assert_owner();
    self.state.hub_chain_id = hub_chain_id;
    self.state.hub_asset_manager = hub_asset_manager;
}

#[payable]
```

```
pub fn change_owner(&mut self, new_owner: AccountId) {
    assert_one_yocto();
    self.assert_owner();
    self.owner = new_owner;
}
```

This forces the use of a Full Access Key (which requires user signature) because Function Access Keys cannot attach deposits, and `assert_one_yocto()` requires exactly 1 yoctoNEAR to be attached. This is a standard NEAR security pattern documented in multiple protocol audits.

Discussion

ibrizsabin

@vendrell46 Thank you . Acknowledged.

vendrell46

Fixed

Issue L-4: Missing event emissions for relevant functions in `asset-manager`, `rate-limit`, and `intent-filler` [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/22

Summary

`asset-manager`, `rate-limit`, and `intent-filler` lack event emissions for critical administrative state-changing operations, while their EVM counterparts properly emit events for the same operations. This prevents off-chain systems from tracking configuration changes and creates an inconsistent security posture across chains.

Vulnerability Detail

These administrative setter functions fail to emit events when critical state changes occur:

asset-manager contract:

```
pub fn set_connection(&mut self, connection: AccountId) {
    self.assert_owner();
    self.state.connection = connection;  //  No event emission
}

pub fn set_rate_limit(&mut self, rate_limit: AccountId) {
    self.assert_owner();
    self.state.rate_limit = rate_limit;  //  No event emission
}

pub fn set_hub_config(&mut self, hub_chain_id: u128, hub_asset_manager: Vec<u8>) {
    self.assert_owner();
    self.state.hub_chain_id = hub_chain_id;
    self.state.hub_asset_manager = hub_asset_manager;  //  No event emission
}

pub fn change_owner(&mut self, new_owner: AccountId) {
    self.assert_owner();
    self.owner = new_owner;  //  No event emission
}
```

rate-limit contract:

```
pub fn set_connection(&mut self, connection: AccountId) {
    self.assert_owner();
    self.state.connection = connection;  //  No event emission
```

```
}

pub fn set_asset_manager(&mut self, asset_manager: AccountId) {
    self.assert_owner();
    self.state.asset_manager = asset_manager;  //  No event emission
}

pub fn set_rate_limit(&mut self, token: String, rate_per_second: u128,
↪  max_allowed_withdrawal: u128) {
    self.assert_owner();
    self.set_rate_limit_inner(token, rate_per_second, max_allowed_withdrawal);  //
     ↪  No event emission
}

pub fn unpause(&mut self) {
    self.assert_owner();
    self.state.paused = false;  //  No event emission
}
```

intent-filler contract:

```
pub fn set_connection(&mut self, connection: AccountId) {
    self.assert_owner();
    self.connection = connection;  //  No event emission
}

pub fn set_hub_config(&mut self, hub_chain_id: u128, hub_intents: Vec<u8>) {
    self.assert_owner();
    self.hub_chain_id = hub_chain_id;
    self.hub_intents = hub_intents;  //  No event emission
}
```

The EVM contracts properly emit events for equivalent operations

Impact

Off-chain monitoring systems, indexers, and user interfaces cannot track relevant configuration changes without repeatedly querying contract state.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/lib.rs#L314-L333

```
pub fn set_rate_limit(&mut self, rate_limit: AccountId) {
    self.assert_owner();
    self.state.rate_limit = rate_limit;
}
```

```
pub fn set_connection(&mut self, connection: AccountId) {
    self.assert_owner();
    self.state.connection = connection;
}

pub fn set_hub_config(&mut self, hub_chain_id: u128, hub_asset_manager: Vec<u8>) {
    self.assert_owner();
    self.state.hub_chain_id = hub_chain_id;
    self.state.hub_asset_manager = hub_asset_manager;
}

pub fn change_owner(&mut self, new_owner: AccountId) {
    self.assert_owner();
    self.owner = new_owner;
}
```

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/rate-limit/src/lib.rs#L215-L229

```
pub fn unpause(&mut self) {
    self.assert_owner();
    self.state.paused = false;
}

pub fn set_asset_manager(&mut self, asset_manager: AccountId) {
    self.assert_owner();
    self.state.asset_manager = asset_manager;
}

pub fn set_connection(&mut self, connection: AccountId) {
    self.assert_owner();
    self.state.connection = connection;
}
```

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/intent-filler/src/lib.rs#L256-L268

```
pub fn set_connection(&mut self, connection: AccountId) {
    self.assert_owner();
    self.connection = connection;
}

pub fn set_hub_config(&mut self, hub_chain_id: u128, hub_intents: Vec<u8>) {
    self.assert_owner();
    require!(hub_chain_id > 0, "hub_chain_id zero");
    require!(!hub_intents.is_empty(), "hub_intents empty");
    self.hub_chain_id = hub_chain_id;
    self.hub_intents = hub_intents;
```

```
}
```

Tool Used

Manual Review

Recommendation

Implement event emissions for all administrative functions using the existing event infrastructure. Define new event types in the events module and emit them after state changes:

Discussion

ibrizsabin

Acknowledged. Thank you.

vendrell46

Fixed

Issue L-5: Rate Limiter set_rate_limit lacks bounds checking [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/26

Summary

The `set_rate_limit` function validates that `rate_per_second != 0` but doesn't validate maximum values. Setting extremely high rates can cause integer overflow in `compute_available`, breaking rate limiting.

Vulnerability Detail

EVM Pattern:

```
function setRateLimit(uint256 ratePerSecond, uint256 maxWithdrawal) external
↪  onlyOwner {
    require(ratePerSecond > 0, "Rate must be positive");
    require(ratePerSecond <= MAX_RATE_PER_SECOND, "Rate too high");
    require(maxWithdrawal > 0, "Max withdrawal must be positive");
    require(maxWithdrawal <= MAX_WITHDRAWAL, "Max withdrawal too high");
    emit RateLimitUpdated(ratePerSecond, maxWithdrawal);
    // ...
}
```

NEAR Implementation:

```
pub fn set_rate_limit(
    &mut self,
    token: String,
    rate_per_second: u128,
    max_allowed_withdrawal: u128,
) {
    self.assert_owner();
    // Only validates rate_per_second != 0
    // No maximum bounds checking
    // No event emission
    self.set_rate_limit_inner(token, rate_per_second, max_allowed_withdrawal);
}

fn set_rate_limit_inner(...) {
    if rate_per_second == 0 {
        env::panic_str("Invalid Ratelimit");
    }
    // No maximum validation
```

```
    // Can set rate_per_second = u128::MAX, causing overflow in compute_available
}
```

Impact

Extremely high rates cause overflow in `compute_available`.

Code Snippet

Tool Used

Manual Review

Recommendation

Consider implementing bounds like an EVM contracts.

Discussion

ibrizsabin

Ya this is an issue. Acknowledged.

vendrell46

Fixed.

Issue L-6: Inconsistent constructor parameter validation across NEAR contracts leads to deployment failure risk [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/28

Summary

The `asset-manager` and `rate-limit` contracts lack constructor parameter validation, while `intent-filler` implements proper validation checks. This inconsistency violates the protocol's own security standards and creates deployment risk where invalid initialization parameters would permanently brick the contracts without any recovery mechanism.

Vulnerability Detail

The protocol establishes a clear security standard in the `intent-filler` contract by validating critical constructor parameters:

```
pub fn new(connection: AccountId, hub_chain_id: u128, hub_intents: Vec<u8>) -> Self
↪ {
    require!(!hub_intents.is_empty(), "hub_intents empty");
    require!(hub_chain_id > 0, "hub_chain_id zero");
    // ... rest of initialization
}
```

However, the `asset-manager` and `rate-limit` contracts do not follow this same defensive practice, accepting potentially invalid values without validation.

asset-manager Constructor:

```
pub fn new(
    connection: AccountId,
    rate_limit: AccountId,
    hub_chain_id: u128,
    hub_asset_manager: Vec<u8>,
) -> Self {
    require!(!env::state_exists(), "Already initialized");
    //  No validation of parameters
    Self {
        state: State {
            connection: connection,
            rate_limit: rate_limit,
            hub_asset_manager: hub_asset_manager,  // Could be empty
            hub_chain_id: hub_chain_id,            // Could be 0
        },
        owner: env::predecessor_account_id(),
```

```
        pending_transfers: LookupMap::new(b"pending_transfers".to_vec()),
        transfer_id_counter: 0,
    }
}
```

rate-limit Constructor:

```
pub fn new(
    asset_manager: AccountId,
    connection: AccountId,
    hub_admin: Vec<u8>,
    hub_manager: Vec<u8>,
    hub_chain_id: u128,
    hub_signers: Vec<Vec<u8>>,
) -> Self {
    //  No validation of any parameters
    let state = State {
        paused: false,
        asset_manager,
        connection,
        hub_admin: hub_admin.into(),      // Could be empty
        hub_manager: hub_manager.into(),  // Could be empty
        hub_chain_id: hub_chain_id.into(), // Could be 0
        hub_signers: hub_signers,         // Could be empty
    };
    Self {
        state,
        rate_limits: LookupMap::new(b"r".to_vec()),
    }
}
```

Deployment Failure Scenarios:

1. **hub_chain_id = 0**: The `recv_message()` function checks `require!(src_chain_id == self.hub_chain_id, "Invalid source chain")`. If `hub_chain_id` is 0, only messages from chain 0 would be accepted, but chain 0 does not exist, rendering the contract unable to receive any cross-chain messages.
2. **Empty hub_asset_manager/hub_admin**: The `recv_message()]`function validates `require!(src_address == self.hub_asset_manager, "Invalid source address")`. If deployed with an empty address, no messages can ever be received since no one can send from an empty address.
3. **Invalid connection account**: The `assert_connection()]` check verifies `predecessor == self.connection`. If deployed with an incorrect connection account, the contract cannot receive messages from the legitimate connection contract.

Impact

Deployment with invalid constructor parameters permanently bricks the contract with no recovery mechanism. The contract becomes unable to receive cross-chain messages, making it completely non-functional. Since NEAR contracts lack built-in upgrade mechanisms, the only resolution is redeployment and migration. The presence of validation in `intent-filler` demonstrates the protocol team recognizes this risk and has established it as a security standard, making the absence in other contracts an inconsistent security posture that violates the protocol's own defensive practices.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/lib.rs#L71-L88

```
pub fn new(
    connection: AccountId,
    rate_limit: AccountId,
    hub_chain_id: u128,
    hub_asset_manager: Vec<u8>,
) -> Self {
    require!(!env::state_exists(), "Already initialized");
    Self {
        state: State {
            connection: connection,
            rate_limit: rate_limit,
            hub_asset_manager: hub_asset_manager,
            hub_chain_id: hub_chain_id,
        },
        owner: env::predecessor_account_id(),
        pending_transfers: LookupMap::new(b"pending_transfers".to_vec()),
        transfer_id_counter: 0,
    }
}
```

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/rate-limit/src/lib.rs#L71-L90

```
pub fn new(
    asset_manager: AccountId,
    connection: AccountId,
    hub_admin: Vec<u8>,
    hub_manager: Vec<u8>,
    hub_chain_id: u128,
    hub_signers: Vec<Vec<u8>>,
) -> Self {
    let state = State {
        paused: false,
```

```
        asset_manager,
        connection,
        hub_admin: hub_admin.into(),
        hub_manager: hub_manager.into(),
        hub_chain_id: hub_chain_id.into(),
        hub_signers: hub_signers,
    };
    Self {
        state,
        rate_limits: LookupMap::new(b"r".to_vec()),
    }
}
```

Tool Used

Manual Review

Recommendation

Apply the same validation pattern used in `intent-filler` to all constructor functions for consistency:

```
// asset-manager
pub fn new(
    connection: AccountId,
    rate_limit: AccountId,
    hub_chain_id: u128,
    hub_asset_manager: Vec<u8>,
) -> Self {
    require!(!env::state_exists(), "Already initialized");
    require!(hub_chain_id > 0, "Invalid hub_chain_id");
    require!(!hub_asset_manager.is_empty(), "Invalid hub_asset_manager");

    Self {
        state: State {
            connection,
            rate_limit,
            hub_asset_manager,
            hub_chain_id,
        },
        owner: env::predecessor_account_id(),
        pending_transfers: LookupMap::new(b"pending_transfers".to_vec()),
        transfer_id_counter: 0,
    }
}

// rate-limit
```

```
pub fn new(
    asset_manager: AccountId,
    connection: AccountId,
    hub_admin: Vec<u8>,
    hub_manager: Vec<u8>,
    hub_chain_id: u128,
    hub_signers: Vec<Vec<u8>>,
) -> Self {
    require!(hub_chain_id > 0, "Invalid hub_chain_id");
    require!(!hub_admin.is_empty(), "Invalid hub_admin");
    require!(!hub_manager.is_empty(), "Invalid hub_manager");
    require!(!hub_signers.is_empty(), "Invalid hub_signers");

    let state = State {
        paused: false,
        asset_manager,
        connection,
        hub_admin: hub_admin.into(),
        hub_manager: hub_manager.into(),
        hub_chain_id: hub_chain_id.into(),
        hub_signers: hub_signers,
    };
    Self {
        state,
        rate_limits: LookupMap::new(b"r".to_vec()),
    }
}
```

Discussion

ibrizsabin

ya this is an issue. Acknowledged.

vendrell46

Fixed

Issue L-7: Function returning Result uses expect() instead of error propagation leading to inconsistent error handling [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/30

Summary

The `Transfer::from_bytes()` function in `asset-manager/src/transfer.rs` declares a `Result<Self, String>` return type but uses `.expect()` internally, causing panics instead of returning errors. This violates the function's contract and makes error handling at call sites ineffective.

Vulnerability Detail

The `from_bytes()` function signature indicates it should handle errors gracefully by returning a `Result`:

```
pub fn from_bytes(bytes: &[u8]) -> Result<Self, String> {
    let transfer: Transfer = rlp::decode(bytes).expect("Failed To Decode Transfer");
    Ok(transfer)
}
```

However, the implementation uses `.expect()` which panics on RLP decoding errors instead of returning `Err(String)`. This violates Rust's error handling conventions where functions returning `Result` should propagate errors, not panic.

The caller in asset-manager/src/lib.rs attempts to handle errors gracefully:

```
let transfer = Transfer::from_bytes(&payload)
    .unwrap_or_else(|_| env::panic_str("Invalid transfer payload"));
```

This error handling is **dead code** because if RLP decoding fails, the panic occurs inside `from_bytes()` before it can return an `Err()` variant. The `.unwrap_or_else()` closure will never execute.

Execution flow:

1. Valid RLP → `rlp::decode()` succeeds → Returns `Ok(transfer)` → Caller receives transfer ✅
2. Invalid RLP → `rlp::decode()` fails → `.expect()` panics → Function never returns → Caller's error handler never executes ❌

Impact

The function's `Result<Self, String>` signature promises error propagation but actually panics, making error handling at call sites unreachable dead code. This violates Rust idioms and creates a misleading contract where developers expect graceful error handling that doesn't exist, reducing code maintainability and making future error recovery logic silently ineffective.

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/transfer.rs#L59-L62

```
pub fn from_bytes(bytes: &[u8]) -> Result<Self, String> {
    let transfer: Transfer = rlp::decode(bytes).expect("Failed To Decode Transfer");
    Ok(transfer)
}
```

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/asset-manager/src/lib.rs#L161-L162

```
let transfer = Transfer::from_bytes(&payload)
    .unwrap_or_else(|_| env::panic_str("Invalid transfer payload"));
```

Tool Used

Manual Review

Recommendation

Replace `.expect()` with proper error propagation using the `?` operator:

```
pub fn from_bytes(bytes: &[u8]) -> Result<Self, String> {
    let transfer: Transfer = rlp::decode(bytes)
        .map_err(|e| format!("Failed to decode transfer: {:?}", e))?;
    Ok(transfer)
}
```

Discussion

ibrizsabin

Acknowledged.

vendrell46

Fixed

Issue L-8: Missing validation and view function in Router whitelist management [RESOLVED]

Source: https://github.com/sherlock-audit/2025-11-sodax-nov-11th/issues/34

Summary

The Router contract's whitelist management functions lack input validation, event emissions, and a view function to check whitelist status, reducing usability and preventing off-chain monitoring.

Vulnerability Detail

The Router contract manages a whitelist that controls which accounts can call `execute_calls()`. However, the whitelist management implementation has several issues:

1. No view function to check whitelist status:

```
pub struct RouterContract {
    pub owner_id: AccountId,
    pub whitelist: LookupSet<AccountId>,
    // ...
}
```

There is no `is_whitelisted(account_id)` function. Users must attempt to call `execute_calls()` and check if it reverts to determine if they are whitelisted.

2. No validation in add_whitelist:

```
#[payable]
pub fn add_whitelist(&mut self, account_id: AccountId) {
    self.assert_owner();
    self.whitelist.insert(account_id);  // Ignores return value
}
```

The function ignores the boolean return value from `LookupSet::insert()`, which indicates whether the account was newly added (`true`) or already existed (`false`). This allows redundant operations that waste gas without feedback.

3. No validation in remove_whitelist:

```
#[payable]
pub fn remove_whitelist(&mut self, account_id: AccountId) {
    self.assert_owner();
    self.whitelist.remove(&account_id);  // Ignores return value
}
```

Similarly, the function ignores the boolean return value from `LookupSet::remove()`, allowing attempts to remove non-existent accounts without feedback.

4. No event emissions:

Neither function emits events, making it impossible to track whitelist changes off-chain for monitoring or auditing purposes.

Impact

- Users cannot query their whitelist status without attempting a state-changing call
- Owner can perform redundant add/remove operations without feedback, wasting gas
- Off-chain systems cannot monitor whitelist changes for security auditing or user notifications
- Poor integration experience compared to standard access control patterns

Code Snippet

https://github.com/sherlock-audit/2025-11-sodax-nov-11th/blob/main/sodax-contracts/near/router/src/lib.rs#L212-L221

Tool Used

Manual Review

Recommendation

Add a view function and improve the management functions:

```
// Add view function
pub fn is_whitelisted(&self, account_id: AccountId) -> bool {
    self.whitelist.contains(&account_id)
}

// Improve add_whitelist with validation and event
#[payable]
pub fn add_whitelist(&mut self, account_id: AccountId) {
    self.assert_owner();
    require!(
        self.whitelist.insert(account_id.clone()),
        "Account already whitelisted"
    );
    env::log_str(&format!("Whitelisted: {}", account_id));
}
```

```
// Improve remove_whitelist with validation and event
#[payable]
pub fn remove_whitelist(&mut self, account_id: AccountId) {
    self.assert_owner();
    require!(
        self.whitelist.remove(&account_id),
        "Account not whitelisted"
    );
    env::log_str(&format!("Removed from whitelist: {}", account_id));
}
```

Discussion

ibrizsabin

Acknowledged. Thank you.

vendrell46

Four issues were raised here, and three have been fixed.

- View function. Added ☒
- `add_whitelist` validation. Added ☒
- `remove_whitelist` validation. Added ☒
- Event emissions. Not added. ☒

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.